

1. Introdução

Quando falamos em processadores gráficos (GPUs), é comum associar imediatamente à ideia de “placa de vídeo para rodar jogo”. Embora isso não esteja errado, essa visão é extremamente limitada frente ao papel que as GPUs desempenham hoje na computação moderna. Uma GPU contemporânea não é apenas um acessório para exibir imagens bonitas na tela; ela é um processador altamente especializado, com uma arquitetura desenhada desde o início para explorar paralelismo em larga escala. Isso significa que, enquanto uma CPU é construída para tomar decisões complexas, responder a interrupções, gerenciar o sistema operacional e executar poucas tarefas por vez com muita flexibilidade, a GPU é construída para executar a mesma operação milhares ou milhões de vezes em dados diferentes, de forma simultânea e altamente eficiente.

Essa diferença de vocação arquitetural faz com que as GPUs sejam imbatíveis em certos tipos de problemas: renderização gráfica em tempo real, processamento de imagens e vídeos, simulações numéricas de grande porte e, mais recentemente, o treinamento e a inferência de modelos de aprendizado de máquina, especialmente redes neurais profundas. O que começou como um hardware dedicado a triângulos, texturas e pixels, hoje se tornou um motor de cálculo paralelo, capaz de sustentar desde jogos AAA até centros de pesquisa em inteligência artificial.

Dentro da disciplina de Arquitetura e Organização de Computadores, estudar GPUs é importante por dois motivos principais. Primeiro, porque elas são um exemplo concreto de como decisões arquiteturais (quantidade de núcleos, hierarquia de memória, modelo de execução, tipo de paralelismo explorado) impactam diretamente o tipo de problema que o hardware resolve bem. Segundo, porque a tendência da computação é cada vez mais heterogênea: sistemas modernos combinam CPU, GPU e, às vezes, outros aceleradores (como TPUs ou NPUs) para atingir desempenho e eficiência que uma CPU isolada jamais conseguiria.

2. Fundamentação: Conceitos e Evolução

Para entender corretamente o que é uma GPU, é útil começar pela comparação com a CPU. A CPU, de forma simplificada, é um processador generalista. Ela possui poucos núcleos relativamente complexos, com grande quantidade de lógica de controle, predição de desvio, execução especulativa e caches sofisticados. Tudo isso existe para que ela consiga lidar bem com fluxos de controle complicados: ifs, loops irregulares, chamadas de função, interrupções de hardware, mudanças de contexto, entre outros. Já a GPU segue a filosofia oposta: abrir mão de parte dessa flexibilidade e complexidade de controle para maximizar a quantidade de unidades aritméticas simples por área de silício. Em outras palavras, sacrifica “inteligência de controle” para ganhar “brute force” aritmético em larga escala.

Do ponto de vista conceitual, a GPU é um processador que implementa um modelo de paralelismo em nível de dados. Em vez de executar instruções diferentes em dados diferentes (como a CPU frequentemente faz), a GPU executa a mesma instrução em muitos dados diferentes ao mesmo tempo. Isso se aproxima da ideia clássica de SIMD (Single Instruction, Multiple Data), mas em GPUs modernas isso é organizado no que se chama de SIMT (Single Instruction, Multiple Threads). Nesse modelo, temos milhares de

threads leves, organizados em grupos, todos avançando em lockstep sob a mesma sequência de instruções, porém operando sobre elementos de dados distintos.

Historicamente, as primeiras placas aceleradoras gráficas dos anos 1990 não eram programáveis no sentido moderno. Elas implementavam um pipeline gráfico fixo, com etapas rígidas: transformação de vértices, iluminação básica, projeção, rasterização e aplicação de textura. O programador apenas configurava parâmetros, mas não definia a lógica de cada etapa. Com o avanço dos jogos e da computação gráfica, surgiu a demanda por efeitos mais complexos e realistas, e as GPUs passaram a incorporar unidades programáveis — os famosos shaders. Inicialmente esses shaders eram separados por tipo (vertex shaders para processar vértices, pixel ou fragment shaders para determinar a cor de cada pixel). Isso já foi uma revolução, pois permitiu que desenvolvedores escrevessem pequenos programas que rodavam diretamente na GPU para controlar como a cena seria desenhada.

O passo seguinte foi a unificação dessas unidades em arquiteturas de “shader unificado”, onde o mesmo bloco de hardware pode executar diferentes tipos de shader conforme a demanda, melhorando a utilização do chip. Paralelamente, pesquisadores começaram a perceber que, se a GPU consegue fazer muitas operações matemáticas simples em larga escala para gráficos, ela também pode ser usada para outros problemas paralelizáveis — por exemplo, multiplicação de matrizes, transformadas rápidas de Fourier, simulações físicas e assim por diante. Nasce então o conceito de GPGPU (General-Purpose computing on Graphics Processing Units).

Esse movimento se consolidou com o surgimento de plataformas como CUDA (da NVIDIA) e, posteriormente, OpenCL, que oferecem modelos de programação explícitos para explorar o paralelismo da GPU em aplicações gerais. A partir desse momento, a GPU deixou definitivamente de ser apenas um “acessório gráfico” e passou a ser tratada como um coprocessador de propósito geral voltado a tarefas paralelas. Mais recentemente, com a explosão do aprendizado profundo, as GPUs se tornaram o componente central de data centers de IA, laboratórios de pesquisa e infraestrutura de computação de alto desempenho.

3. Funcionamento e Arquitetura Interna

Em termos de organização interna, uma GPU moderna é composta por vários blocos de processamento idênticos, frequentemente chamados de SMs (Streaming Multiprocessors, no caso da NVIDIA) ou Compute Units (na AMD). Cada um desses blocos contém dezenas ou centenas de núcleos simples, além de registradores, memória compartilhada e unidades funcionais especializadas. Em vez de ter poucos núcleos muito sofisticados, como a CPU, a GPU tem muitos núcleos simples agrupados em SMs, o que é ideal para repetir uma mesma operação sobre grandes conjuntos de dados, como todos os pixels de uma imagem ou todos os neurônios de uma camada de rede neural.

O modelo de programação típico de uma GPU é baseado em milhares de threads. Esses threads não são “pesados” como os threads de sistema operacional na CPU; são entidades leves gerenciadas diretamente pela GPU, organizadas em blocos e grade (grid). Cada bloco de threads é atribuído a um SM, que executa esses threads em grupos menores chamados warps (no jargão da NVIDIA) ou wavefronts (na AMD). Um warp

costuma agrupar 32 ou 64 threads que executam a mesma instrução ao mesmo tempo em dados diferentes. É aqui que o modelo SIMT se concretiza: a unidade de controle envia uma instrução, e todas as lanes (os “mini-núcleos” do warp) a executam simultaneamente em seus respectivos dados.

Uma consequência interessante dessa organização é o problema da divergência de controle. Como todos os threads de um warp compartilham o mesmo fluxo de instruções, se dentro de um warp alguns threads entram em um branch (por exemplo, o “if” é verdadeiro para alguns dados e falso para outros), a GPU precisará serializar essas execuções, desativando temporariamente determinados threads enquanto executa o caminho alternativo. Isso reduz a eficiência daquele warp. Portanto, uma parte importante da programação para GPU é tentar organizar os dados e a lógica de forma que os threads de um mesmo warp sigam caminhos semelhantes, minimizando divergências.

Outro aspecto crucial é a hierarquia de memória. A GPU possui uma memória global (a VRAM) com grande capacidade e largura de banda, mas com latência relativamente alta. Para esconder essa latência, a GPU utiliza uma combinação de alto paralelismo (mantendo muitos warps prontos para execução, de modo que, enquanto um warp espera dados da memória, outro possa ser executado) com memórias mais rápidas próximas ao SM. Dentro de cada SM, existe uma memória compartilhada (shared memory), que funciona como um “scratchpad” extremamente rápido para comunicação entre threads de um mesmo bloco, além de caches L1 e registradores. Em nível superior, há um cache L2 que atende todos os SMs. O desempenho de um código de GPU depende fortemente de como as leituras e escritas de memória são organizadas: acessos coalescidos, onde threads vizinhos acessam endereços de memória consecutivos, são muito mais eficientes do que acessos dispersos e aleatórios.

No contexto gráfico tradicional, essa arquitetura é utilizada para implementar o pipeline de renderização. A CPU prepara os dados da cena (geometria, texturas, parâmetros de iluminação) e envia comandos para a GPU através de APIs como DirectX, OpenGL ou Vulkan. A GPU então passa esses dados pelo pipeline, que inclui etapas como transformação de coordenadas (do espaço do mundo para o espaço da câmera e depois para o espaço de tela), clipping de polígonos, rasterização (conversão de primitivas geométricas, como triângulos, em fragmentos/pixels), execução de shaders que determinam a cor final de cada fragmento e, por fim, escrita no framebuffer que será exibido no monitor. Cada etapa é altamente paralelizável, pois cada vértice ou cada pixel pode ser processado em paralelo com muitos outros.

Para computação de propósito geral, o funcionamento é semelhante, mas em vez de vértices e pixels, trabalhamos com dados arbitrários (vetores, matrizes, tensores). O programador lança kernels de GPU — pequenos programas — para serem executados por milhares de threads sobre grandes blocos de dados. A GPU, então, distribui esse trabalho entre os SMs e mantém o máximo possível de threads ativos, preenchendo os “buracos” causados por latência de memória ou dependências de dados com outros threads prontos para executar. A chave da eficiência está em alimentar constantemente a GPU com trabalho paralelizável e organizar os dados de forma amigável à sua hierarquia de memória.

4. Vantagens e Benefícios

O grande diferencial das GPUs é a capacidade de oferecer uma quantidade gigantesca de operações de ponto flutuante por segundo (FLOPs) para workloads que se encaixam bem no modelo de paralelismo em nível de dados. Em tarefas como multiplicação de grandes matrizes, convoluções em redes neurais, aplicações de filtros em imagens ou simulações numéricas discretizadas, a mesma operação é repetida um número enorme de vezes com pequenas variações de dados. Isso é exatamente o tipo de coisa que uma GPU faz melhor que qualquer outro processador de uso geral.

Do ponto de vista de desempenho, uma única GPU de alto desempenho pode substituir dezenas de CPUs em termos de throughput para certas aplicações, especialmente em inteligência artificial e computação científica. Além disso, a relação desempenho por Watt costuma ser mais favorável em GPUs quando o problema é altamente paralelizável. Isso significa que, para um determinado nível de desempenho, uma solução baseada em GPU pode consumir menos energia do que uma fazenda equivalente de CPUs. Esse fator é crítico em data centers, onde energia e refrigeração representam uma parcela significativa dos custos operacionais.

Na prática, isso se traduz em benefícios concretos: jogos com gráficos mais ricos e realistas rodando em tempo real; softwares de edição de vídeo capazes de renderizar e exportar projetos complexos em minutos em vez de horas; modelos de aprendizado de máquina que antes demoravam semanas para treinar em CPUs, hoje podem ser treinados em dias ou horas com conjuntos de GPUs; pesquisadores de áreas como física, química, biologia computacional e climatologia conseguindo simular fenômenos com granularidade antes inviável. Em muitas dessas áreas, a GPU não é apenas uma otimização; ela é o fator que torna o problema tratável na prática.

Outro benefício importante é a offload de tarefas da CPU. Ao delegar à GPU toda a carga pesada de cálculo paralelizável, a CPU pode se concentrar na orquestração geral do sistema: lógica de jogo, entrada de usuário, comunicação de rede, tarefas do sistema operacional, entre outras. Isso melhora o desempenho global e a responsividade da máquina. Em sistemas embarcados, como carros autônomos ou dispositivos de realidade aumentada, o uso coordenado de CPU + GPU permite realizar processamento de visão computacional e IA em tempo real, algo que seria impraticável apenas com CPUs tradicionais.

5. Desafios e Limitações

Apesar de toda a potência das GPUs, elas não são uma solução mágica para todos os problemas. O primeiro grande desafio é de natureza algorítmica e conceitual: nem todo problema é naturalmente paralelizável ao ponto de justificar o uso de uma GPU. Problemas com fluxos de controle muito irregulares, muitas dependências entre etapas ou forte necessidade de acesso aleatório à memória, em geral, não se adaptam bem ao modelo SIMT e à hierarquia de memória das GPUs. Nesses casos, a CPU continua sendo a melhor opção, e tentar “forçar” o problema numa GPU pode resultar em código mais complexo e, ainda por cima, mais lento.

O segundo desafio está na programação. Escrever código eficiente para GPU não é apenas traduzir um algoritmo de CPU para CUDA ou OpenCL; muitas vezes é necessário repensar a estrutura do algoritmo, reorganizar dados, reduzir divergência de branches,

cuidar da coalescência de acessos à memória e fazer uso inteligente das diferentes camadas de memória (registradores, memória compartilhada, caches, global). Esse processo exige uma curva de aprendizado considerável, tanto em termos de modelo mental quanto de domínio das ferramentas.

Há também limitações físicas e de infraestrutura. GPUs de alto desempenho consomem muita energia e geram muito calor, exigindo fontes de alimentação robustas e soluções de refrigeração mais sofisticadas. Em ambientes de data center, o consumo agregado de dezenas ou centenas de GPUs é um fator crítico, tanto pelo custo de energia quanto pelos desafios térmicos. Além disso, a comunicação entre CPU e GPU, normalmente via barramento PCI Express, é muito mais lenta do que a comunicação interna da GPU com sua VRAM. Isso significa que, se o programa ficar constantemente transferindo grandes volumes de dados entre CPU e GPU, a vantagem de desempenho pode ser severamente prejudicada.

Outra limitação está relacionada ao custo. GPUs poderosas, especialmente as voltadas para data center e IA, possuem preço elevado e forte demanda no mercado. Isso pode dificultar o acesso a esse tipo de recurso por laboratórios menores ou desenvolvedores individuais. Além disso, a rápida obsolescência tecnológica faz com que gerações de GPUs fiquem defasadas em poucos anos, tanto em termos de performance quanto de suporte de software.

Finalmente, há um desafio arquitetural: à medida que as GPUs se tornam mais complexas, as fronteiras entre CPU, GPU e outros aceleradores começam a se misturar. Integrar esses componentes de forma eficiente, tanto em hardware quanto em software, é um problema de pesquisa ativa. Tecnologias como memórias unificadas, interconexões de alta velocidade entre processadores e modelos de programação heterogêneos estão em desenvolvimento constante, mas ainda não são triviais de explorar ao máximo.

6. Perspectivas e Considerações Finais

Olhando para o futuro, é difícil imaginar a computação de alto desempenho sem a presença de GPUs ou de alguma evolução conceitual delas. A demanda por processamento paralelo tende a aumentar, impulsionada por aplicações de inteligência artificial cada vez mais grandes, simulações de fenômenos complexos e experiências multimídia mais imersivas. Nesse cenário, a filosofia arquitetural da GPU — muitos núcleos simples, altíssima largura de banda de memória e foco em paralelismo em nível de dados — continua fazendo muito sentido.

Uma tendência clara é a maior integração entre CPU e GPU. Em muitos sistemas já vemos processadores que combinam ambos no mesmo chip (APUs, SoCs em dispositivos móveis, consoles de videogame), reduzindo a latência de comunicação e simplificando o compartilhamento de memória. Em data centers, surgem soluções em que múltiplas GPUs são interconectadas por barramentos dedicados de altíssima velocidade, formando verdadeiros “supercomputadores modulares”. Paralelamente, aceleradores ainda mais especializados, como TPUs para operações de redes neurais ou NPUs em dispositivos móveis, complementam o papel das GPUs, mostrando que a computação caminha rumo a um ecossistema heterogêneo, onde diferentes tipos de processador atacam diferentes partes do problema.

Para a arquitetura e organização de computadores, as GPUs servem como um estudo de caso interessante de como decisões sobre paralelismo, hierarquia de memória e modelo de execução moldam as capacidades de uma máquina. Elas reforçam a ideia de que não existe um “processador perfeito” para tudo; existem arquiteturas mais adequadas para determinadas classes de problemas. Ao compreender a lógica por trás das GPUs, o estudante consegue enxergar com mais clareza por que certas aplicações exigem um tipo específico de hardware e por que a programação paralela se torna cada vez mais central na formação em computação.

Em resumo, os processadores gráficos evoluíram de componentes periféricos, dedicados exclusivamente a desenhar pixels na tela, para peças centrais da infraestrutura de computação contemporânea. Ao dominar seus conceitos fundamentais, seu funcionamento interno e suas limitações, o grupo não apenas estará preparado para o seminário, mas também terá uma base sólida para entender muitos dos avanços tecnológicos que vêm moldando a computação nos últimos anos — especialmente no campo da inteligência artificial e da computação de alto desempenho.